College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID721001: Mobile Application Development
Level 7, Credits 15
Project

Assessment Overview

In this **individual** assessment, you will develop two mobile games using **Unity**.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Implement and publish complete, non-trivial, industry-standard mobile applications following sound architectural and code-quality standards.
2. Identify relevant use cases for a mobile computing scenario and incorporate them into an effective user experience design.
3. Follow industry standard software engineering practice in the design of mobile applications.

Assessments

Assessment	Weighting	Due Date	Learning Outcomes
Practical	20%	Sunday 9th November at 11.59 PM	1, 2
Project	80%	Sunday 9th November at 11.59 PM	1, 2, 3

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Sunday 9th November** at **11.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID721001: Mobile Application Development**.

Submission

You **must** submit all files via **GitHub Classroom**. Here is the URL to the repository you will use for your submission – https://classroom.github.com/a/Mq0MeQ_x. If you do not have not one, create a **.gitignore** and add the ignored files in this resource - https://raw.githubusercontent.com/github/gitignore/main/Node.gitignore.

Create a branch called **project**. The latest files in the **project** branch will be used to mark against the marking rubric. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **12.00 AM**.

Authenticity

All parts of your submitted assessment **must** be completely your work. The use of **AI tools** is not permitted in this assessment. You need to demonstrate to the course lecturer that you can meet the learning outcomes for this assessment. You **must** complete the Academic Integrity Declaration form and put this in the root of your repository. Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments are not applicable in **ID721001: Mobile Application Development**.

Instructions

Functionality - Learning Outcomes 1, 2 (20%)

- Game v1 where the user interacts using basic gestures.
- Game v2 with **ten features** of your choice (this can build on v1).
- Create an **APK** for both v1 and v2.

Code Quality and Best Practices - Learning Outcomes 1, 3 (35%)

- Write clean, readable, and maintainable code.
- Use modular, reusable components and avoid code duplication.
- Keep code simple and easy to understand.
- Ensure the games are responsive across different screen sizes and orientations.
- Optimise performance by managing resources efficiently.

Version Control - Learning Outcome 3 (5%)

- Regular commits are made throughout development, demonstrating meaningful progress.
- Commit messages are clear and descriptive.
- The repository shows a clean and logical commit history that reflects the development process of the games.

Testing - Learning Outcome 3 (10%)

- Test the games with three different users for each version (six users total), observing how they navigate and use features.
- Collect written feedback on usability issues, confusing UI and bugs.
- Write a brief report summarising the user testing, including:
  - Tester information (anonymised).
  - Tasks performed.
  - Positive feedback received.
  - Areas for improvement identified.
  - Bugs or errors found.
  - Suggestions on how feedback could be used to improve the games (no requirement to implement).

Reflection - Learning Outcomes 1, 3 (5%)

- Write a short reflection (300-500 words) on your development process and learning throughout the project, including:
  - Challenges you faced and how you overcame them.
  - What you would do differently in future projects.
  - New skills, tools or practices you learned during the project.

Presentation - Learning Outcome 2 (5%)

- Record a 5-10 minute screen recording of your games.
- Demonstrate the v1 and v2 features.
- Make sure the video is clear and easy to follow.
- Submit a link to the presentation in your repository's **README.md** file.

Marking Rubric

Functionality - Learning Outcomes 1, 2 (20%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Game v1 - Basic Gestures (8%)	Game fully functional with user interaction through basic gestures (tap, swipe, drag, pinch, etc.). Gestures are responsive, smooth and intuitive. Game mechanics are well-implemented and polished. Core gameplay is engaging and complete.	Game functional with basic gesture interaction. Gestures mostly responsive with minor issues. Game mechanics implemented with some polish. Core gameplay present.	Basic game present with gesture interaction partially implemented. Some responsiveness issues. Game mechanics basic. Core gameplay incomplete or limited.	Game non-functional or gestures severely limited/broken. Poor responsiveness. Game mechanics missing or broken. No complete gameplay.
Game v2 - Ten Features (8%)	Ten distinct features fully implemented and functional. Features are creative, well-integrated and significantly enhance gameplay. Features work together cohesively. Clear progression from v1 or standalone excellence.	8-10 features implemented with minor issues. Features mostly integrated and add value to gameplay. Good progression or standalone quality.	6-8 features present with functionality gaps. Basic integration. Some features add limited value.	Fewer than 6 features or non-functional. Poor integration. Features don't enhance gameplay or are incomplete.
APK Creation (4%)	Both v1 and v2 APKs successfully created, installable and fully functional on Android devices. Professional build configuration. Games run smoothly with no crashes.	Both APKs created with minor issues. Mostly functional after installation. Minor performance issues or occasional bugs.	Both APKs created but with functionality or installation issues. Games run but with significant bugs or crashes.	APKs missing, won't install, only one version created, or games crash immediately.

Code Quality and Best Practices - Learning Outcomes 1, 3 (35%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Code Readability and Maintainability (9%)	Code is exceptionally clean and readable with consistent formatting. Meaningful variable, function and class names. Well-organised file and script structure. Comprehensive comments where needed. Follows C# and Unity conventions.	Code is clean and readable with good naming conventions. Organised structure with some helpful comments. Minor inconsistencies. Mostly follows conventions.	Code is mostly readable. Some unclear naming or inconsistent formatting. Basic organisation present. Limited comments.	Code is difficult to read with poor naming conventions. Inconsistent or chaotic structure. No comments. Doesn't follow conventions.
Modularity and Reusability (9%)	Excellent use of modular, reusable scripts and components. Classes and functions are well-abstracted and single-purpose. No code duplication. DRY principles followed throughout. Proper use of Unity component system.	Good modular structure with reusable components. Minimal code duplication. Most code follows DRY principles. Adequate use of Unity components.	Some modular components present. Some code duplication exists. Partial adherence to DRY principles. Basic Unity component usage.	Little modularity. Significant code duplication. Poor component abstraction. Improper Unity component usage.
Code Simplicity and Clarity (8%)	Code is simple, elegant and easy to understand. Complex logic is well-explained. Appropriate use of Unity features and C# without over-engineering. Clear separation of concerns.	Code is generally simple and clear. Most logic is easy to follow with minor complex sections. Reasonable use of Unity features.	Code works but contains unnecessarily complex sections. Some logic is difficult to follow. Basic Unity feature usage.	Code is overly complex or convoluted. Difficult to understand the implementation. Misuse of Unity features.

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Responsive Design (5%)	Games are fully responsive across different screen sizes and orientations. UI scales and adapts seamlessly. Excellent handling of aspect ratios. Touch inputs work correctly on all screen sizes.	Games mostly responsive with minor issues on some screen sizes or orientations. UI scales adequately. Touch inputs mostly work.	Basic responsive design present. Some screens adapt but with layout or scaling issues. Touch inputs work on most devices.	Poor or no responsive design. UI breaks on different screen sizes or orientations. Touch inputs fail on some devices.
Performance and Resource Management (4%)	Excellent resource management. Efficient use of GameObjects and components. Proper memory management and object pooling. Optimised physics and rendering. Smooth frame rates (60 FPS) with no stuttering.	Good resource management. Mostly efficient implementations. Adequate optimization. Minor frame rate drops. Good overall performance.	Basic resource management. Some inefficient implementations. Performance acceptable but not optimised. Noticeable frame drops.	Poor resource management. Inefficient use of resources. Performance issues present. Frequent frame drops, stuttering or crashes. Memory leaks.

Version Control - Learning Outcome 3 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Commit History and Messages (5%)	Regular, meaningful commits throughout development. Clear, descriptive commit messages. Logical progression showing development process. Clean commit history in project branch. Proper .gitignore usage.	Regular commits with mostly clear messages. Commit history shows good development progression with minor gaps. Good .gitignore usage.	Commits present but irregular or clustered. Some commit messages unclear. Basic development history visible. Basic .gitignore present.	Few commits or poor timing. Unclear messages (e.g., ”update”, ”fix”). No clear development progression. No or improper .gitignore.

Testing - Learning Outcome 3 (10%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
User Testing Process (4%)	Six users tested (three per version). Well-structured testing sessions with clear tasks relevant to mobile game interaction. Comprehensive written feedback collected covering usability, UI, controls and bugs. Professional documentation.	5-6 users tested. Testing sessions conducted with mostly clear tasks. Good written feedback collected covering most areas including mobile-specific issues.	3-4 users tested. Basic testing conducted. Some written feedback collected but incomplete or lacking mobile gaming focus.	Fewer than 3 users tested or testing poorly documented. Minimal or no written feedback. No focus on mobile gaming aspects.
Testing Report Quality (6%)	Comprehensive report with all required sections. Tester information anonymised. Tasks clearly documented. Thorough analysis of positive feedback, improvement areas and bugs. Excellent suggestions for improvements with clear rationale specific to mobile gaming.	Good report covering all required sections. Most elements well-documented. Adequate analysis of feedback and bugs. Good improvement suggestions relevant to mobile games.	Report present covering most sections. Basic documentation of testing. Some analysis present. Limited improvement suggestions.	Report missing, incomplete or significantly lacking detail. Poor documentation. No analysis or improvement suggestions.

Reflection - Learning Outcomes 1, 3 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Reflection Quality (5%)	Thoughtful reflection (300-500 words) covering all required points. Clear articulation of challenges, solutions and learning. Demonstrates deep understanding and self-awareness about mobile game development process.	Good reflection covering all required points. Adequate discussion of challenges and learning with some depth. Within word count.	Reflection present covering most points. Brief or superficial discussion. May be slightly outside word count.	Reflection missing, incomplete or significantly outside word count. Lacks depth or doesn't cover required points.

Presentation - Learning Outcome 2 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Video Demonstration (5%)	Clear 5-10 minute recording demonstrating all v1 and v2 features. Well-paced, professional presentation. Easy to follow with good audio/visual quality. Shows games running on device or emulator. Clearly demonstrates gesture interactions. Link submitted correctly.	Good recording within time limit. Most features demonstrated. Generally clear with minor audio/visual issues. Shows gameplay. Link submitted.	Recording present but may be too short/long. Some features demonstrated. Quality acceptable but could be clearer.	Recording missing, significantly outside time limit, unclear, or doesn't demonstrate key features. Poor quality. Link missing.